Project In Practice

From **Unknonwn Unknowns** to **Known Knowns** through **Known Unknowns**.

Identifying Unknown Unknowns

When we start the project, we are in the "**unknown unknown**" state: in other words, we don't even know what we don't know yet.

- We need tools and rules for the transition from **unknown unknowns** to **known unknowns** as quickly as possible.

Iterative Process

We have two iterations (sprints) in the **course** projects.

- The first iteration is aim to identify unknown unknowns as quickly as possible.
    - We need to change features/requirements if we know that the features are not feasible through prototypes and MVPs.
- The second iteration is to build the features based on our understanding.

Prototypes

We should build prototypes when we haven't built similar software before.

- Prototypes are for checking feasibility and identifying unknown unknowns as quickly as possible.
- We don't have make architectures or designs, not even tests.
- Even in this case, we should make data models.
- We can change anything later.

MVPs

MVP (Most Viable Product) is the software that just works with minimum features.

- In the next iterations, we refactor design add features based on the MVP.
- MVP should have all the necessary designs, tests, and documents.

Aim for High-Quality

As professional software engineers, we should aim for building high-quality software: well architectured and designed

- The software that is easy to **find and fix bugs**.
- The software that is easy to **add features**.

We can focus on solving real problems only when we have this architecture/design.

Roles and Responsibilites

In a class project, each student plays the role with responsibilities.

Professor (as Manager)

- **Manage People & Politics:** Navigate complexity and team dynamics.
- **Report Upward:** Communicate with high-level officials (ASE/CS Committee).
- **Evaluate Teams:** Assess tech leads and team members.
- **Support Problem-Solving:** Provide guidance and resources to resolve issues.

In a real-world, managers do this important job:

- **Decide Careers:** Handle promotions, raises, hiring, and firing.

Team Leaders (Tech Leads)

- **Manage Team Complexity:** Solve problems by organizing people and tasks.
- **Report to Managers:** Keep leadership informed.
- **Set Direction:** Define goals and schedules.
- **Drive Progress:** Ensure team members move forward.
- **Communicate Upward:** Report team progress to managers.
- **Manage Artifacts:** Code, tests and documents

- **Preside the Weekly Meetings:** Team leaders should present their weekly progress.
    - When leaders cannot preside the meetings, they should delegate the presentation to one of the team members.
- **Lead the Presentations:** They should lead the presentations.

Team Members (As Senior Level Software Engineers)

- **Tackle Complexity:** Solve problems through design and implementation.
- **Design Architecture:** Make data models and manage how the data is flowing through modules.
- **Make Design:** Make modules and interfaces
- **Build & Test:** Write code and tests when needed.

- **Raise Issues:** Signal problems early to get them resolved.
- **Mentor Others:** Support junior developers and interns.

> In the age of AI, there is (practically) no junior-level software engineering jobs; each student should be the senior-level problem solver.

Team Rules

No Surprises

> Unexpected problems break trust more than difficult problems.

1. **Communicate Early:** Raise issues as soon as they appear.
2. **Be Transparent:** Share progress, blockers, and risks openly.
3. **Set Expectations:** Keep managers and teammates informed.
4. **Avoid Last-Minute Shocks:** No one should be blindsided.

Show No Emotions

> Professionalism means staying calm, even under pressure.

1. **Stay Calm:** Don’t let frustration or excitement cloud communication.
2. **Focus on Facts:** Discuss data and solutions, not feelings.
3. **Maintain Composure:** Handle conflicts without visible anger or stress.
4. **Build Trust:** A steady presence reassures the team and clients.

Be a Professional

> Professionalism is about actions and results, not titles.

1. **Deliver on Promises:** Do what you say, on time.
2. **Respect Others:** Treat everyone with fairness and courtesy.
3. **Take Responsibility:** Own mistakes and learn from them.
4. **Strive for Quality:** Aim for excellence in every task.

Understand Others before Being Understood

> Listening deeply builds trust faster than speaking first.

1. **Seek Context:** Learn the origin of the problem and who's involved.
2. **Listen Actively:** Let others feel heard before offering input.
3. **Empathize First:** Acknowledge concerns and perspectives.
4. **Respond Thoughtfully:** Solutions land better when

Project Recommendations

Identify Unknown Unknows Quickly

> We can make predictable plans only when we identify all the unknown unknowns.

1. **Start Early:** Begin quickly to reveal what you don’t know.
2. **Prototype First:** Test feasibility before the MVP to ensure you’re solving the right problem.
3. **Seek Clarity:** Work hard to gain a clear vision as a problem solver.

Make it a Game, Enjoy Small Victories

> Big achievements are just a collection of small wins stacked together.

1. **Gamify the Work:** Treat challenges like levels to beat.
2. **Celebrate Progress:** Every small win builds momentum.
3. **Stay Motivated:** Fun fuels persistence through obstacles.
4. **Use All Course Materials:** Treat homework, exams, and quizzes as opportunities to practice problem-

Make it Work, then Make it Better

> A rough solution that works is more valuable than a perfect one that never ships.

1. **Start Simple:** Deliver something functional first.
2. **Prove Value:** Show it solves the problem before refining.
3. **Iterate Later:** Improve design, speed, and polish after validation.
4. **Avoid Perfection Trap:** Progress beats endless tweaking.

Effective first, efficient later

> Planning the fastest trip to New York is useless if you really need to be in Los Angeles.

1. **Focus on Impact:** Solve the right problem before worrying about speed or cost.
2. **Validate Direction:** Early solutions may be rough, but they prove value and feasibility.
3. **Refine for Efficiency:** Once effectiveness is clear, streamline and optimize.
4. **Avoid Premature Optimization:** The first goal is to ship the MVP, not perfect it.

Think in Systems, Build the System

> Strong systems outlast quick fixes and scale beyond individual effort.

1. **Vertical Slice First:** Build the system to deliver end-to-end value instead of isolated layers.
2. **Sustainable Foundations:** Design systems and processes with growth and change in mind.
3. **Think Solutions in Systems:** Let solution patterns emerge naturally within the system.
4. **System as Accelerator:** Once built, the system makes every next project faster and easier.

System-Driven Processes

> A good system creates processes that run and improve themselves.

1. **Self-Improving:** The process continuously makes the process better.
2. **Reduce Mental Burden:** Free people from micromanagement and memory load.
3. **Consistency:** Ensure reliable results regardless of who executes the task.
4. **Scalability:** Allow teams to grow without adding complexity.